

```
!pip install plotly kaleido --quiet

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.ticker as mtick
import seaborn as sns
import plotly.express as px
import plotly.graph_objects as go
from plotly.subplots import make_subplots
import warnings
warnings.filterwarnings('ignore')

# Display settings
pd.set_option('display.max_columns', 50)
pd.set_option('display.float_format', '{:.2f}'.format)
print('Libraries loaded successfully')
```

Libraries loaded successfully

```
# Load all required CSVs (relative paths – no local drive references)
orders    = pd.read_csv('olist_orders_dataset.csv')
reviews   = pd.read_csv('olist_order_reviews_dataset.csv', engine='python')
customers = pd.read_csv('olist_customers_dataset.csv')
products  = pd.read_csv('olist_products_dataset.csv')
items     = pd.read_csv('olist_order_items_dataset.csv')
translate = pd.read_csv('product_category_name_translation.csv')

print('Dataset shapes:')
for name, df in [('orders', orders), ('reviews', reviews), ('customers', customers),
                 ('products', products), ('items', items), ('translate', translate)]:
    print(f' {name:12s}: {df.shape[0]:>7,} rows x {df.shape[1]} cols')
```

```
Dataset shapes:
orders      : 99,441 rows x 8 cols
reviews     : 99,224 rows x 7 cols
customers   : 99,441 rows x 5 cols
products    : 32,951 rows x 9 cols
items       : 112,650 rows x 7 cols
translate   :    71 rows x 2 cols
```

```
# Deduplicate reviews – keep first review per order_id
reviews['review_creation_date'] = pd.to_datetime(reviews['review_creation_date'])
reviews_deduped = (
    reviews.sort_values('review_creation_date')
    .drop_duplicates(subset='order_id', keep='first')
)

# Join: Orders ← Reviews ← Customers
master = (
    orders
    .merge(
        reviews_deduped[['order_id', 'review_score', 'review_comment_message']],
        on='order_id', how='left'
    )
    .merge(
        customers[['customer_id', 'customer_state', 'customer_city']],
        on='customer_id', how='left'
    )
)
```

```
# Acceptance Criteria: No row duplication
assert len(master) == len(orders), f'Row mismatch! {len(master)} vs {len(orders)}'
print(f' Master dataset: {len(master):,} rows (matches orders table – no duplicates)')
print(f' Columns: {list(master.columns)}')
master.head(3)
```

Master dataset: 99,441 rows (matches orders table – no duplicates)

Columns: ['order_id', 'customer_id', 'order_status', 'order_purchase_timestamp', 'order_approved_at', 'order_delivered_carrier_date', 'order_delivered_customer_date', 'order_estimated_delivery_date']

	order_id	customer_id	order_status	order_purchase_t
0	e481f51cbdc54678b7cc49136f2d6af7	9ef432eb6251297304e76186b10a928d	delivered	2017-10-02
1	53cdb2fc8bc7dce0b6741e2150273451	b0830fb4747a6c6d20dea0b8c802d7ef	delivered	2018-07-24
2	47770eb9100c2d0c44946d9cf07ec65d	41ce2a54c0b03bf3443c3d931a367089	delivered	2018-08-08

Next steps: [Generate code with master](#)

[New interactive sheet](#)

```
# Parse date columns
date_cols = [
    'order_purchase_timestamp',
    'order_approved_at',
    'order_delivered_carrier_date',
    'order_delivered_customer_date',
    'order_estimated_delivery_date'
]
for col in date_cols:
    master[col] = pd.to_datetime(master[col])

# Filter: delivered orders only (exclude canceled, unavailable, etc.)
undelivered_statuses = ['canceled', 'unavailable']
delivered = master[master['order_status'] == 'delivered'].copy()
flagged = master[master['order_status'].isin(undelivered_statuses)].copy()

print(f'Order status breakdown:')
print(master['order_status'].value_counts().to_string())
print(f'\n Working with {len(delivered):,} delivered orders')
print(f' {len(flagged):,} orders flagged (canceled/unavailable) – excluded from delay analysis')
```

Order status breakdown:

```
order_status
delivered      96478
shipped         1107
canceled         625
unavailable     609
invoiced         314
processing       301
created           5
approved         2
```

Working with 96,478 delivered orders

1,234 orders flagged (canceled/unavailable) – excluded from delay analysis

```
# Calculate Days_Difference
delivered['days_difference'] = (
    delivered['order_estimated_delivery_date'] - delivered['order_delivered_customer_date']
).dt.days
```

Drop rows where actual delivery date is missing

```
delivered = delivered.dropna(subset=['order_delivered_customer_date', 'order_estimated_delivery_date'])
```

```
# Classify delivery status
def classify_delivery(days):
    if days >= 0:
        return 'On Time'
    elif days >= -5:
        return 'Late'
    else:
        return 'Super Late'

delivered['delivery_status'] = delivered['days_difference'].apply(classify_delivery)

# Summary statistics
status_counts = delivered['delivery_status'].value_counts()
status_pct = delivered['delivery_status'].value_counts(normalize=True).mul(100).round(1)
summary = pd.DataFrame({'Count': status_counts, 'Percentage': status_pct})
print('Delivery Status Summary:')
print(summary.to_string())
print(f'\nMedian days difference: {delivered["days_difference"].median():.1f} days')
print(f'Mean days difference: {delivered["days_difference"].mean():.1f} days')
```

Delivery Status Summary:

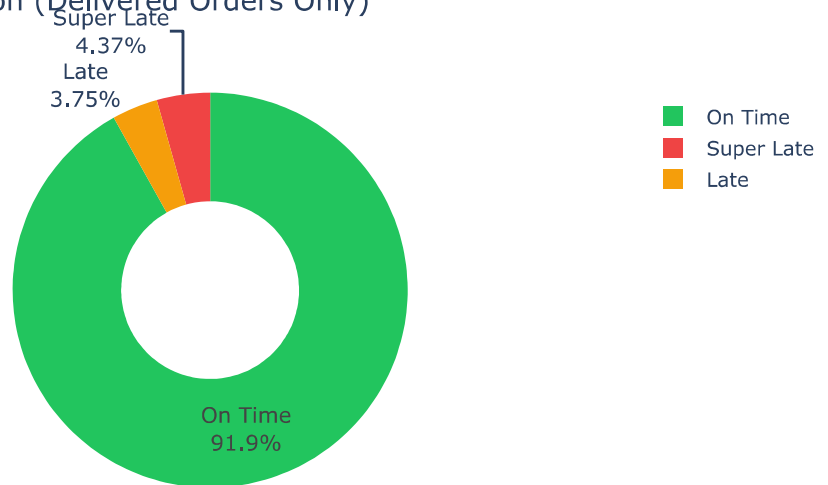
	Count	Percentage
delivery_status		
On Time	88644	91.90
Super Late	4211	4.40
Late	3615	3.70

Median days difference: 11.0 days
Mean days difference: 10.9 days

```
# Visualize delivery status distribution
color_map = {'On Time': '#22c55e', 'Late': '#f59e0b', 'Super Late': '#ef4444'}

fig = px.pie(
    names=status_counts.index,
    values=status_counts.values,
    color=status_counts.index,
    color_discrete_map=color_map,
    title='Delivery Status Distribution (Delivered Orders Only)',
    hole=0.45
)
fig.update_traces(textinfo='percent+label', textfont_size=13)
fig.update_layout(height=420, showlegend=True)
fig.show()
```

Delivery Status Distribution (Delivered Orders Only)



```
# Aggregate late rate per state
state_stats = (
    delivered.groupby('customer_state')
    .agg(
        total_orders=('order_id', 'count'),
        late_orders=('delivery_status', lambda x: (x != 'On Time').sum()),
        avg_review=('review_score', 'mean'),
        avg_delay=('days_difference', 'mean')
    )
    .reset_index()
)

state_stats['pct_late'] = (state_stats['late_orders'] / state_stats['total_orders'] * 100).round(1)
state_stats = state_stats.sort_values('pct_late', ascending=False)

print('Top 10 States by % Late Deliveries:')
print(state_stats[['customer_state', 'total_orders', 'pct_late', 'avg_review']].head(10).to_string(index=False))
```

```
Top 10 States by % Late Deliveries:
customer_state  total_orders  pct_late  avg_review
AL              397          23.90         3.86
MA              717          19.70         3.83
PI              476          16.00         3.99
CE             1279          15.30         3.94
SE              335          15.20         3.91
BA             3256          14.00         3.93
RJ            12350          13.50         3.96
TO              274          12.80         4.15
PA              946          12.40         3.91
RR              41          12.20         3.90
```

```
# Bar chart - % late by state
fig = px.bar(
    state_stats,
    x='customer_state', y='pct_late',
    color='pct_late',
    color_continuous_scale='RdYlGn_r',
    title='% Late Deliveries by Brazilian State',
    labels={'pct_late': '% Late', 'customer_state': 'State'},
    text='pct_late'
```

```

)
fig.update_traces(texttemplate='%{text:.1f}%', textposition='outside')
fig.update_layout(
    height=500,
    xaxis_tickangle=-45,
    coloraxis_showscale=False,
    plot_bgcolor='white'
)
fig.show()

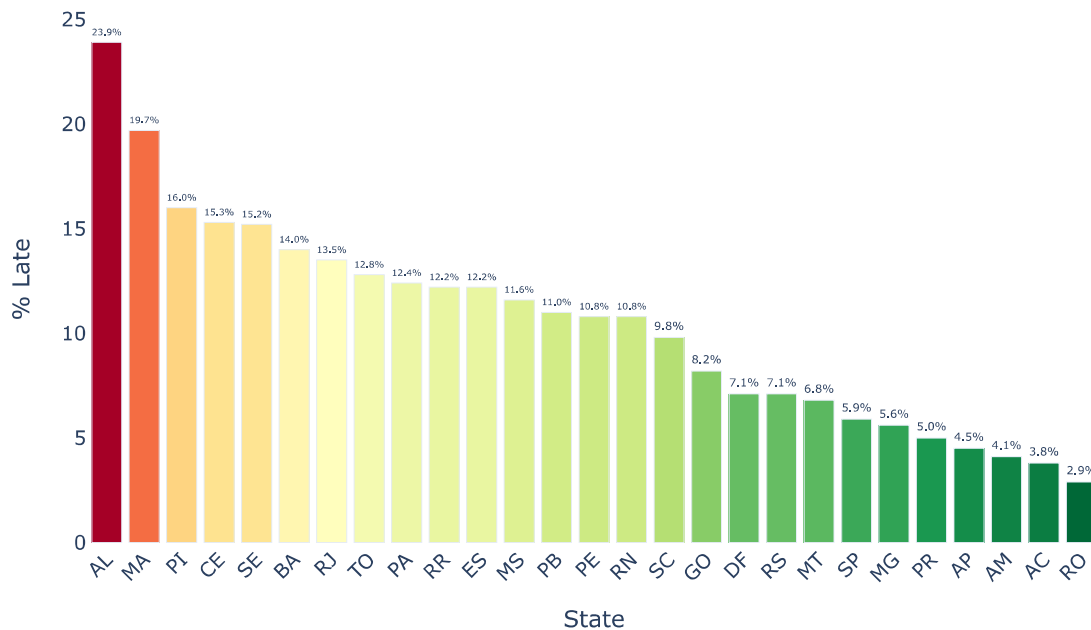
# Identify remote vs hub states
remote_states = ['AM', 'RR', 'AP', 'PA', 'AC', 'RO', 'TO', 'MA']
hub_states     = ['SP', 'RJ', 'MG', 'PR', 'SC', 'RS']

remote_avg = state_stats[state_stats['customer_state'].isin(remote_states)]['pct_late'].mean()
hub_avg     = state_stats[state_stats['customer_state'].isin(hub_states)]['pct_late'].mean()
national    = state_stats['pct_late'].mean()

print(f'\n📊 Late Delivery Rate Comparison:')
print(f' Remote/Northern states avg: {remote_avg:.1f}%')
print(f' Hub/Southern states avg:   {hub_avg:.1f}%')
print(f' National average:           {national:.1f}%')
print(f'\n Remote states are ~{remote_avg/hub_avg:.1f}x more likely to receive late deliveries')

```

% Late Deliveries by Brazilian State



📊 Late Delivery Rate Comparison:
 Remote/Northern states avg: 9.1%
 Hub/Southern states avg: 7.8%
 National average: 10.5%

Remote states are ~1.2x more likely to receive late deliveries

```

# Average review score by delivery status
sentiment_by_status = (
    delivered.groupby('delivery_status')['review_score']
    .agg(['mean', 'count', 'std'])

```

```

.reset_index()
.rename(columns={'mean': 'avg_score', 'count': 'n_orders', 'std': 'std_dev'})
)
# Sort for logical display
status_order = ['On Time', 'Late', 'Super Late']
sentiment_by_status['delivery_status'] = pd.Categorical(
    sentiment_by_status['delivery_status'], categories=status_order, ordered=True
)
sentiment_by_status = sentiment_by_status.sort_values('delivery_status')

print('Average Review Score by Delivery Status:')
print(sentiment_by_status.to_string(index=False))

fig = px.bar(
    sentiment_by_status,
    x='delivery_status', y='avg_score',
    color='delivery_status',
    color_discrete_map={'On Time': '#22c55e', 'Late': '#f59e0b', 'Super Late': '#ef4444'},
    title='Average Review Score: On Time vs Late vs Super Late',
    labels={'avg_score': 'Avg Review Score (1-5)', 'delivery_status': 'Delivery Status'},
    text='avg_score',
    error_y='std_dev'
)
fig.update_traces(texttemplate='%{text:.2f}', textposition='outside')
fig.update_layout(height=450, yaxis_range=[0, 5.5], showlegend=False, plot_bgcolor='white')
fig.show()

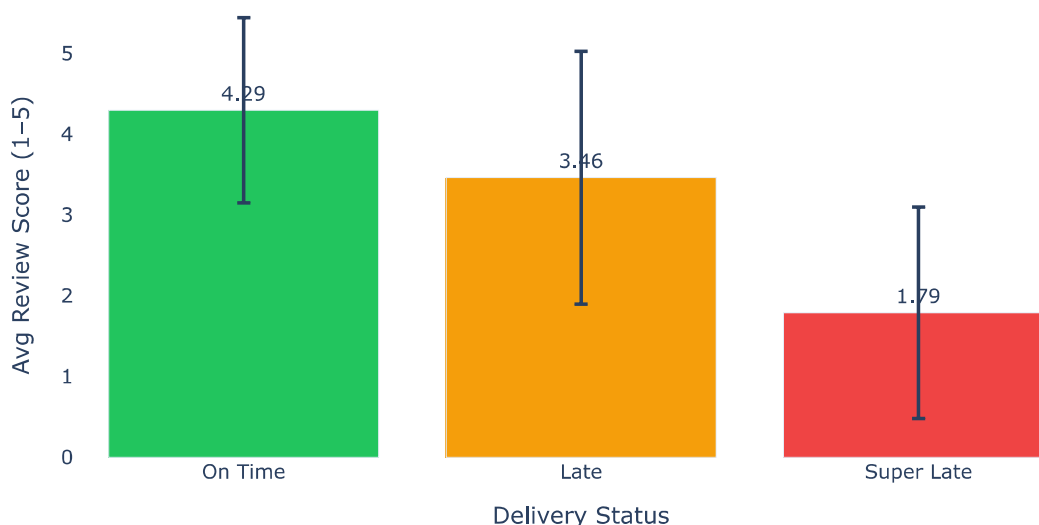
```

```

Average Review Score by Delivery Status:
delivery_status avg_score n_orders std_dev
On Time        4.29      88163    1.15
Late           3.46      3568     1.56
Super Late     1.79       4093     1.31

```

Average Review Score: On Time vs Late vs Super Late



```

# Scatter: Days Late vs Review Score (sampled for performance)
delivered['days_late'] = (-delivered['days_difference']).clip(lower=0)

sample = delivered.sample(min(8000, len(delivered)), random_state=42)

```

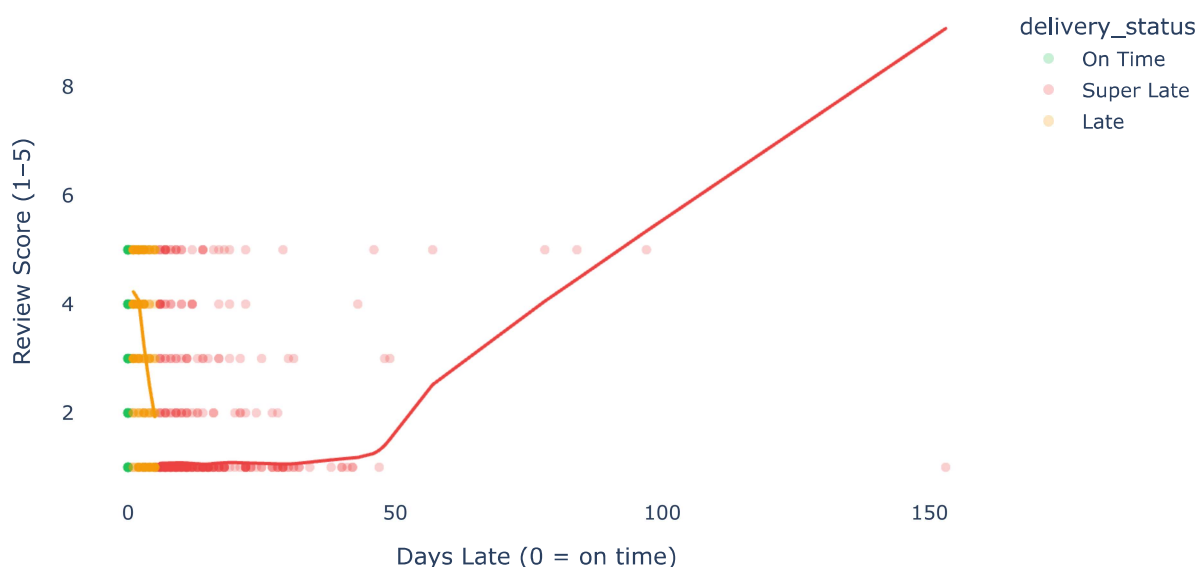
```

fig = px.scatter(
    sample,
    x='days_late', y='review_score',
    color='delivery_status',
    color_discrete_map={'On Time': '#22c55e', 'Late': '#f59e0b', 'Super Late': '#ef4444'},
    trendline='lowess',
    opacity=0.25,
    title='Days Late vs. Customer Review Score (LOWESS trend)',
    labels={'days_late': 'Days Late (0 = on time)', 'review_score': 'Review Score (1-5)'}
)
fig.update_layout(height=480, plot_bgcolor='white')
fig.show()

# Correlation coefficient
corr = delivered[['days_late', 'review_score']].corr().iloc[0, 1]
print(f'\nPearson correlation (days_late vs review_score): {corr:.3f}')
print(' (Negative = more late days → lower score, as expected)')

```

Days Late vs. Customer Review Score (LOWESS trend)



Pearson correlation (days_late vs review_score): -0.276
 (Negative = more late days → lower score, as expected)

```

# Join items → products → translation
items_with_cat = (
    items
    .merge(products[['product_id', 'product_category_name']], on='product_id', how='left')
    .merge(translate, on='product_category_name', how='left')
)

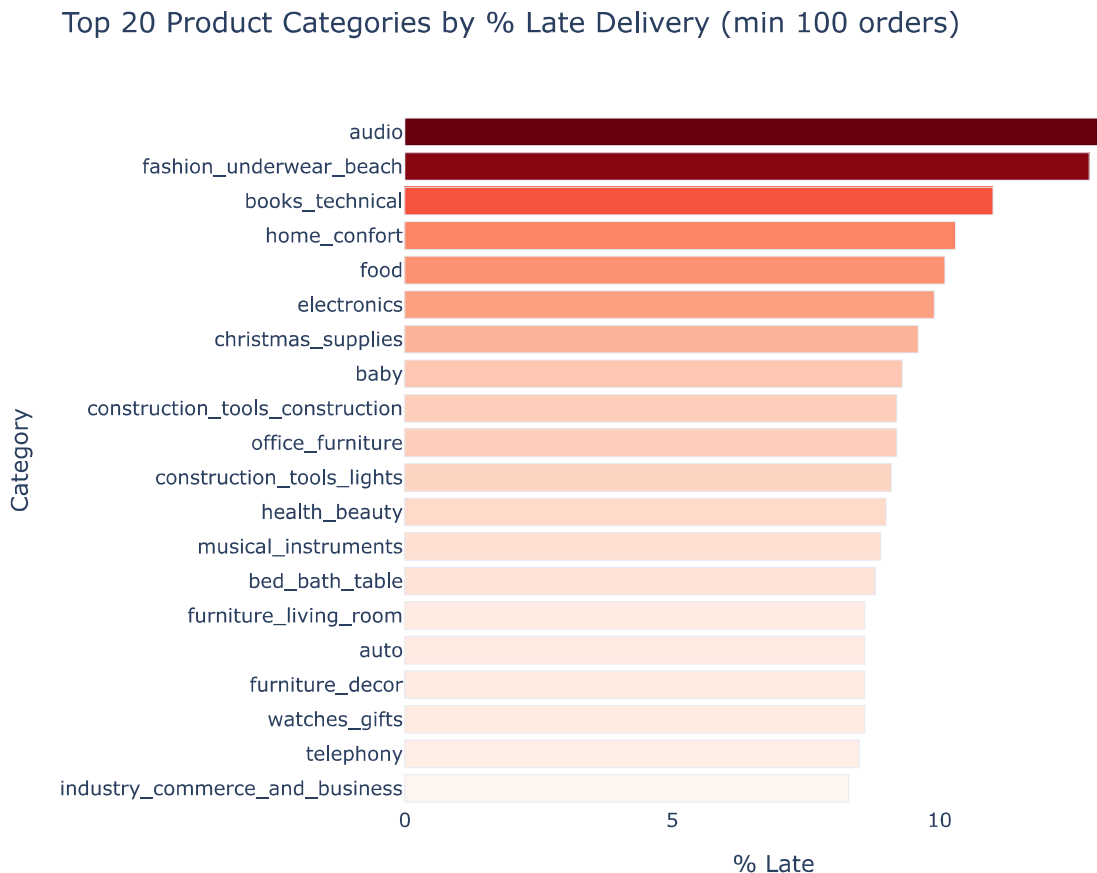
# One primary category per order (first item's category)
cat_per_order = (
    items_with_cat.groupby('order_id')['product_category_name_english']
    .first()
    .reset_index()
)

delivered = delivered.merge(cat_per_order, on='order_id', how='left')

```

```
# Late rate by English category (min 100 orders for statistical significance)
cat_stats = (
    delivered.groupby('product_category_name_english')
    .agg(
        total=('order_id', 'count'),
        late=('delivery_status', lambda x: (x != 'On Time').sum()),
        avg_review=('review_score', 'mean')
    )
    .reset_index()
)
cat_stats['pct_late'] = (cat_stats['late'] / cat_stats['total'] * 100).round(1)
cat_stats = cat_stats[cat_stats['total'] >= 100].sort_values('pct_late', ascending=False)

fig = px.bar(
    cat_stats.head(20),
    x='pct_late', y='product_category_name_english',
    orientation='h',
    color='pct_late',
    color_continuous_scale='Reds',
    title='Top 20 Product Categories by % Late Delivery (min 100 orders)',
    labels={'pct_late': '% Late', 'product_category_name_english': 'Category'}
)
fig.update_layout(height=600, coloraxis_showscale=False, plot_bgcolor='white', yaxis={'autorange': ' '})
fig.show()
```



```
fig = make_subplots(
    rows=1, cols=2,
    subplot_titles=(
        'Distribution of Delivery Slack (Estimated - Actual)'
    )
)
```

```

        'Monthly On-Time Rate Trend'
    )
)

# Left: Histogram of days_difference
fig.add_trace(
    go.Histogram(
        x=delivered['days_difference'],
        nbinsx=80,
        marker_color='#6366f1',
        opacity=0.8,
        name='Delivery Slack'
    ),
    row=1, col=1
)
fig.add_vline(x=0, line_dash='dash', line_color='red',
              annotation_text='On-Time Boundary', row=1, col=1)

# Right: Monthly on-time rate trend
delivered['purchase_month'] = delivered['order_purchase_timestamp'].dt.to_period('M').astype(str)
monthly = (
    delivered.groupby('purchase_month')
    .agg(ontime_rate=('delivery_status', lambda x: (x == 'On Time').mean() * 100))
    .reset_index()
    .sort_values('purchase_month')
)

fig.add_trace(
    go.Scatter(
        x=monthly['purchase_month'],
        y=monthly['ontime_rate'],
        mode='lines+markers',
        line=dict(color='#22c55e', width=2),
        marker=dict(size=6),
        name='On-Time Rate %'
    ),
    row=1, col=2
)

fig.update_layout(
    height=440,
    showlegend=False,
    plot_bgcolor='white',
    title_text='ETA Bias & Trend Analysis'
)

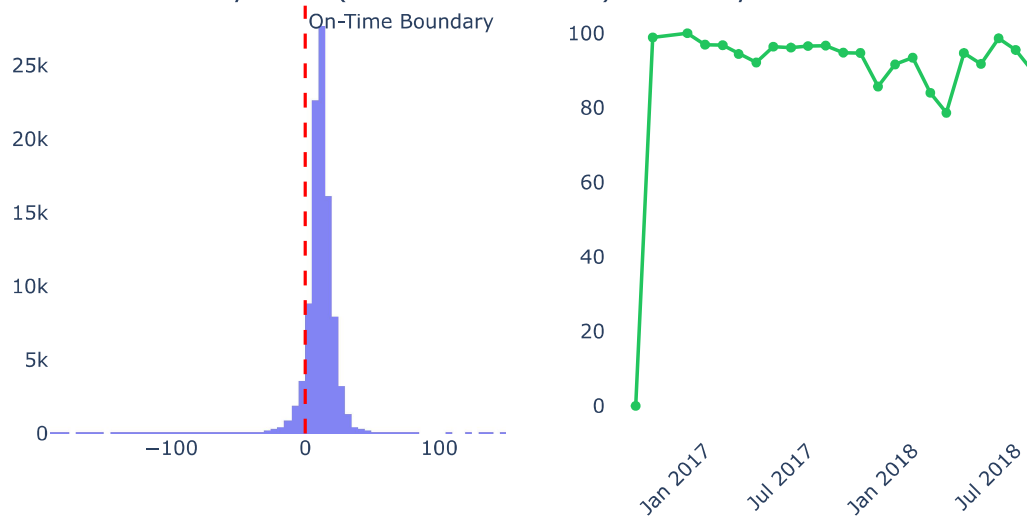
fig.update_xaxes(tickangle=-45, row=1, col=2)
fig.show()

skewness = delivered['days_difference'].skew()
pct_late_overall = (delivered['delivery_status'] != 'On Time').mean() * 100
print(f'Distribution skewness: {skewness:.2f}')
print(f'Overall % late: {pct_late_overall:.1f}%')
if skewness < 0:
    print('⚠️ Left-skewed: estimates are OPTIMISTIC – the CEO is right, we are over-promising.')
else:
    print('✅ Right-skewed: estimates are PESSIMISTIC – we typically under-promise.')

```

ETA Bias & Trend Analysis

Distribution of Delivery Slack (Estimated – Actual) Monthly On-Time Rate Trend



Distribution skewness: -2.02

Overall % late: 8.1%

⚠ Left-skewed: estimates are OPTIMISTIC – the CEO is right, we are over-promising.

```
print('=' * 60)
print(' VERIDI LOGISTICS – DELIVERY AUDIT EXECUTIVE SUMMARY')
print('=' * 60)

total_delivered = len(delivered)
pct_ontime      = (delivered['delivery_status'] == 'On Time').mean() * 100
pct_late_       = (delivered['delivery_status'] == 'Late').mean() * 100
pct_super_late  = (delivered['delivery_status'] == 'Super Late').mean() * 100
avg_review_late = delivered[delivered['delivery_status'] != 'On Time']['review_score'].mean()
avg_review_ot   = delivered[delivered['delivery_status'] == 'On Time']['review_score'].mean()
worst_state     = state_stats.iloc[0]

print(f'\n Total delivered orders analyzed : {total_delivered:,}')
print(f' On-Time rate                        : {pct_ontime:.1f}%')
print(f' Late rate                             : {pct_late_:.1f}%')
print(f' Super Late rate (>5 days)              : {pct_super_late:.1f}%')
print(f'\n Avg review – On Time orders            : {avg_review_ot:.2f} / 5.00')
print(f' Avg review – Late orders                : {avg_review_late:.2f} / 5.00')
print(f' Score drop from late delivery           : {avg_review_ot - avg_review_late:.2f} points')
print(f'\n Worst performing state                  : {worst_state["customer_state"]} ({worst_state["pct_lat')
print(f' Remote states avg late rate              : {remote_avg:.1f}%')
print(f' Hub states avg late rate                 : {hub_avg:.1f}%')
print(f'\n ETA distribution skewness               : {skewness:.2f}')
print('\n KEY FINDING:')
print(f' Late deliveries reduce review scores by ~{avg_review_ot - avg_review_late:.1f} points.')
print(f' Remote/northern states are {remote_avg/hub_avg:.1f}x more likely to be affected.')
print('=' * 60)
```

```
=====
VERIDI LOGISTICS – DELIVERY AUDIT EXECUTIVE SUMMARY
=====
```

```
Total delivered orders analyzed : 96,470
On-Time rate                     : 91.9%
Late rate                         : 3.7%
```

```
Super Late rate (>5 days)      : 4.4%

Avg review – On Time orders    : 4.29 / 5.00
Avg review – Late orders       : 2.57 / 5.00
Score drop from late delivery  : 1.73 points

Worst performing state         : AL (23.9% late)
Remote states avg late rate    : 9.1%
Hub states avg late rate       : 7.8%

ETA distribution skewness      : -2.02
```